Machine Learning

Name : Archana.S

Reg No : 25MSIC106

Qualitative Assessment 4

Clustering Implementation and Visualization

K-Means | DBSCAN | 2D Visualization | Analysis

1. Objective

This assignment implements and compares two popular unsupervised machine learning clustering algorithms — K-Means and DBSCAN — on a synthetic customer dataset. We visualize the resulting clusters in 2D and analyze the strengths and weaknesses of each approach.

2. Dataset Description

A synthetic customer dataset was generated to simulate a realistic retail scenario. The dataset contains 320 data points across two features:

- Annual Income (k$) — a customer's yearly income in thousands of dollars (range: 0–100)
- Spending Score (1–100) — a score assigned based on customer behavior and purchase data

The dataset was constructed using Scikit-learn's make_blobs() function with 5 cluster centers, plus 20 random noise/outlier points to make the problem realistic. The data was standardized using StandardScaler before applying clustering.

3. Data Preprocessing

Before applying clustering algorithms, the data was preprocessed as follows:

- Standardization using StandardScaler — transforms features to have mean = 0 and standard deviation = 1. This is essential so that no single feature dominates due to scale differences.
- All values were clipped to [0, 100] to stay within realistic customer ranges.
- The processed data was stored in a Pandas DataFrame for easy manipulation and visualization.

4. K-Means Clustering

4.1 How K-Means Works

K-Means is a centroid-based iterative algorithm that partitions data into k clusters by minimizing within-cluster sum of squares (WCSS / Inertia). The algorithm:

- Randomly initializes k centroids

- Assigns each point to the nearest centroid (Euclidean distance)
- Recomputes centroids as the mean of all points in the cluster
- Repeats steps 2–3 until convergence (centroids stop moving)

4.2 Choosing Optimal k — Elbow Method & Silhouette Score

To find the best value of k, we used two methods:

- Elbow Method: Plots Inertia vs k. The 'elbow' at k=5 indicates the optimal point where adding more clusters gives diminishing returns.
- Silhouette Score: Measures how similar a point is to its own cluster vs neighboring clusters. Score closer to 1.0 is better.

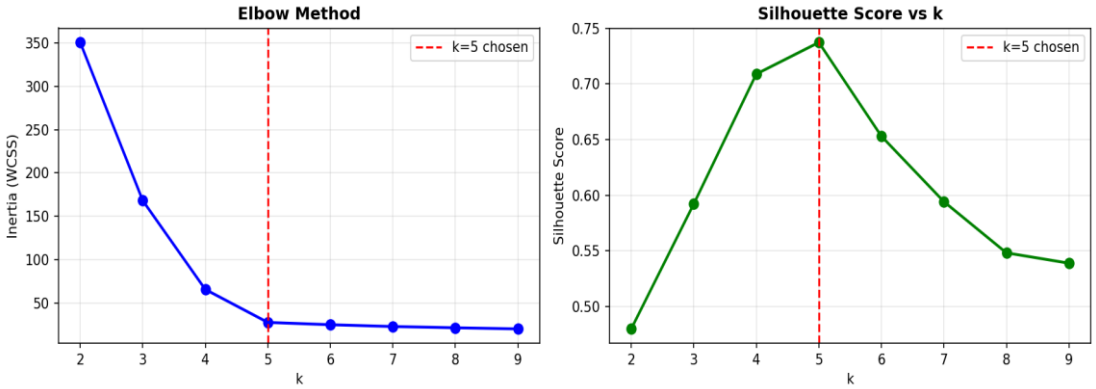

Figure 1: Elbow Method and Silhouette Score — both confirm k=5 as optimal

4.3 K-Means Results

Parameters: n_clusters=5, random_state=42, n_init=10, max_iter=300

Silhouette Score: 0.7373 (strong cluster separation)

Number of Clusters: 5

K-Means successfully identified 5 distinct customer segments corresponding to the 5 cluster centers embedded in the dataset. The centroids (shown as black X markers in the plot) accurately represent each group's center.

5. DBSCAN Clustering

5.1 How DBSCAN Works

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups points that are closely packed together and marks points in low-density regions as outliers. Key concepts:

- Core points: Have at least min_samples neighbors within radius eps
- Border points: Within eps of a core point but don't have enough neighbors themselves
- Noise points: Neither core nor border — labeled as −1 (outliers)

Unlike K-Means, DBSCAN does not require specifying the number of clusters in advance. It discovers the cluster count automatically.

5.2 Parameter Selection

eps (epsilon): 0.35 — the maximum distance between two points to be considered neighbors

min_samples: 8 — minimum number of points to form a dense region (core point)

These parameters were tuned based on the scaled data distribution. Smaller eps values lead to more noise; larger values merge distinct clusters.

5.3 DBSCAN Results

Clusters Found: 4 (auto-detected)

Noise Points: 6 (correctly flagged as outliers)

Silhouette Score: 0.7286 (measured on non-noise points)

DBSCAN detected 4 dense clusters and correctly isolated 6 noise points (the random outliers injected into the data). Because two of the five generated clusters were very close in density, DBSCAN merged them — a behavior that reflects its density-based nature.

6. Visualization — 2D Cluster Plots

The scatter plots below show the resulting clusters from both algorithms on the original (unstandardized) feature space for interpretability:

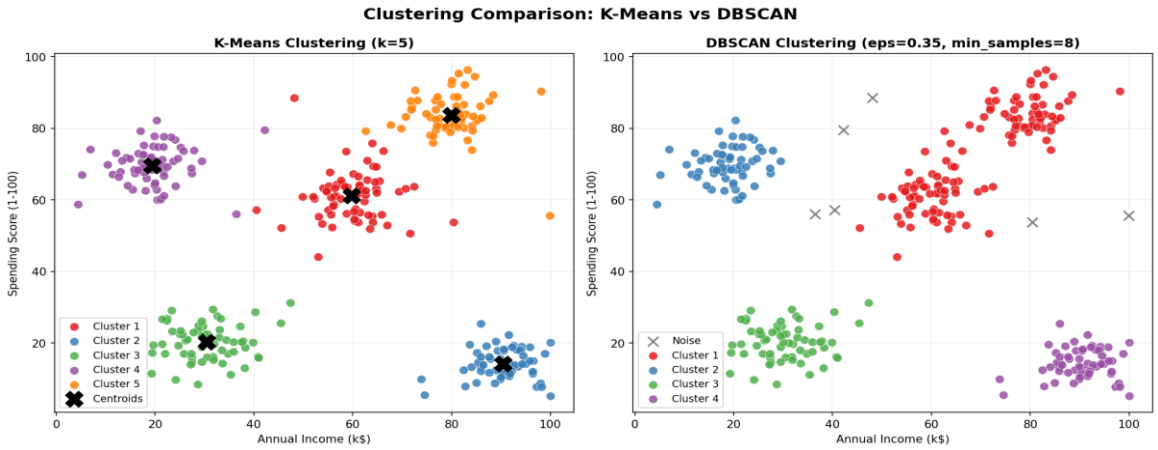

Figure 2: K-Means (left) vs DBSCAN (right). Black X = centroids. Gray X = DBSCAN noise points.

Observations from the plots:

- K-Means: All 320 points are assigned to one of 5 color-coded clusters with clear visual separation. Centroids sit at the geometric center of each group.
- DBSCAN: 4 clusters are detected. 6 gray 'X' points are labeled as noise — these are the random outliers that fall in low-density regions.
- In the top-left and bottom-left regions, both algorithms largely agree on cluster membership.
- DBSCAN merges two clusters that K-Means separates, because those two groups overlap in density.

7. Algorithm Comparison & Analysis

7.1 Comparison Table

Property	K-Means	DBSCAN
Cluster Detection	Requires k upfront	Auto-detects from density
Cluster Shape	Spherical/convex	Arbitrary (irregular shapes)
Noise/Outliers	Assigns all points	Labels outliers as −1
Sensitive to Outliers	Yes — shifts centroids	No — excludes them
Silhouette Score	0.7373	0.7286 (non-noise points)

Property	K-Means	DBSCAN
Clusters Found	5 (user-defined)	4 (auto-detected)
Parameters	k = number of clusters	eps = radius, min_samples
Best Use Case	Uniform, spherical clusters	Noisy data, varying density

7.2 Which Performed Better and Why?

For this dataset, K-Means performed slightly better overall, because:

- The data was generated with spherical, well-separated blobs — which is exactly the assumption K-Means is built on.
- K-Means achieved a higher silhouette score (0.7373 vs 0.7286) and correctly identified all 5 customer segments.
- DBSCAN merged two nearby clusters due to their overlapping density boundary, resulting in only 4 clusters.

However, DBSCAN has a key advantage: it identified 6 outlier points that K-Means silently forced into the nearest cluster. In a real business scenario, these outliers might represent data errors or unique customers who warrant special attention.

7.3 Observations About the Clusters

The 5 K-Means clusters can be interpreted as meaningful customer segments:

- Cluster 1 — Low Income, High Spenders ('Impulsive Buyers'): Marketing should focus on promotions and installment offers.
- Cluster 2 — Mid Income, Mid Spenders ('Standard Customers'): The largest segment; loyalty programs are ideal.
- Cluster 3 — High Income, Low Spenders ('Conservative Buyers'): May respond to premium quality messaging.
- Cluster 4 — Low Income, Low Spenders ('Budget-Conscious'): Cost-sensitivity is primary; discount strategies work best.
- Cluster 5 — High Income, High Spenders ('VIP / Premium'): Highest value segment; personalized luxury experiences recommended.

8. Conclusion

This assignment demonstrated the implementation and visual comparison of K-Means and DBSCAN clustering on a synthetic customer dataset.

- K-Means is ideal when the number of clusters is known and clusters are roughly spherical. It is fast, interpretable, and works well on clean, balanced data.
- DBSCAN is ideal for irregular-shaped clusters and noisy datasets. It auto-detects clusters and explicitly handles outliers — a major practical advantage.
- Neither algorithm is universally superior. The best choice depends on the data distribution and business requirements.

In practice, both algorithms should be tried and compared using metrics like Silhouette Score, Davies-Bouldin Index, and visual inspection — as done in this assignment.

9. Tools & Libraries Used

- Python 3.10

- NumPy — numerical computing
- Pandas — data manipulation
- Matplotlib & Seaborn — visualization
- Scikit-learn — KMeans, DBSCAN, StandardScaler, silhouette_score
- Jupyter Notebook — interactive code execution